

Code Reference: builder-rich-future-sections.js

A source-level explanation with function details, file communication, variables, endpoints, and debugging notes.

Item	Explanation
File	builder-rich-future-sections.js
Purpose	Builder-side helper code for richer future portfolio sections and section defaults.
Lines	410
Size	21,340 bytes
Talks to	Mostly local to its own feature area.
Functions	13
Variables/constants	42
API mentions	0
Signals	17

How this file participates in the system

This generated section reads the actual file and points out line numbers, declarations, endpoints, events, source excerpts, and debugging cues.

Important implementation signals

Item	Explanation
Rich text at line 82	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: <button type="button" data-rich-action="font-small">Small</button>
Rich text at line 83	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: <button type="button" data-rich-action="font-normal">Normal</button>
Rich text at line 84	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: <button type="button" data-rich-action="font-large">Large</button>
Rich text at line 97	Rich editor behavior, formatting, paste handling, or content sanitization. Evidence: data-placeholder="Write the overview here. You can paste images directly into this area."
Browser DOM at line 125	Builder or website interface behavior in the browser/Electron renderer. Evidence: if (editor.dataset.richEditor === "pending-description") {
Browser DOM at line 139	Builder or website interface behavior in the browser/Electron renderer. Evidence: if (editor.dataset.richEditor === "pending-description") {
Browser DOM at line 193	Builder or website interface behavior in the browser/Electron renderer. Evidence: document.querySelector("#pending-editor")?.scrollIntoView({ behavior: "smooth", block: "nearest" });
Browser DOM at line 204	Builder or website interface behavior in the browser/Electron renderer. Evidence: const type = button.dataset.openEditor;
Browser DOM at line 205	Builder or website interface behavior in the browser/Electron renderer. Evidence: const mode = button.dataset.mode "add";
Browser DOM at line 206	Builder or website interface behavior in the browser/Electron renderer. Evidence: const index = button.dataset.index;
Browser DOM at line 208	Builder or website interface behavior in the browser/Electron renderer. Evidence: const section = (project.sections []).find((item) => item.id === button.dataset.sectionId);
Browser DOM at line 213	Builder or website interface behavior in the browser/Electron renderer. Evidence: sectionId: button.dataset.sectionId,
Browser DOM at line 222	Builder or website interface behavior in the browser/Electron renderer. Evidence: const section = (project.sections []).find((item) => item.id === button.dataset.sectionId);
Browser DOM at line 226	Builder or website interface behavior in the browser/Electron renderer. Evidence: sectionId: button.dataset.sectionId,
Cloudflare or AI at line 238	Public AI assistant, Worker deployment, or model-provider fallback behavior. Evidence: // their richDescription field back to the project model. Fallback to

Item	Explanation
Browser DOM at line 245	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>const richEditor = form.querySelector("[data-rich-editor='pending-description']");</code>
Browser DOM at line 393	Builder or website interface behavior in the browser/Electron renderer. Evidence: <code>document.addEventListener("contextmenu", (event) => {</code>

Important constants and variables

Item	Explanation
text	Line 34: <code>const text = plainTextFromRich(rich).trim();</code>
label	Line 73: <code>const label = editor?.type === "custom-section" ? "Section overview" : "Subsection overview";</code>
folder	Line 74: <code>const folder = pendingEditorFolder(editor);</code>
originalPopulateRichEditor	Line 108: <code>const originalPopulateRichEditor = typeof populateRichEditor === "function" ? populateRichEditor : null;</code>
originalSaveRichEditorToProject	Line 109: <code>const originalSaveRichEditorToProject = typeof saveRichEditorToProject === "function" ? saveRichEditorToProject : null;</code>
originalOpenPendingEditor	Line 110: <code>const originalOpenPendingEditor = typeof openPendingEditor === "function" ? openPendingEditor : null;</code>
originalOpenEditorFromButton	Line 111: <code>const originalOpenEditorFromButton = typeof openEditorFromButton === "function" ? openEditorFromButton : null;</code>
originalRenderPendingEditor	Line 112: <code>const originalRenderPendingEditor = typeof renderPendingEditor === "function" ? renderPendingEditor : null;</code>
originalRenderSectionContent	Line 113: <code>const originalRenderSectionContent = typeof renderSectionContent === "function" ? renderSectionContent : null;</code>
originalSavePendingEditor	Line 114: <code>const originalSavePendingEditor = typeof savePendingEditor === "function" ? savePendingEditor : null;</code>
originalRenderCustomSection	Line 115: <code>const originalRenderCustomSection = typeof renderCustomSection === "function" ? renderCustomSection : null;</code>
originalParseProjectForPortfolio	Line 116: <code>const originalParseProjectForPortfolio = typeof parseProjectForPortfolio === "function" ? parseProjectForPortfolio : null;</code>
originalParsedSectionHasContent	Line 117: <code>const originalParsedSectionHasContent = typeof parsedSectionHasContent === "function" ? parsedSectionHasContent : null;</code>
originalPreviewSectionHasRenderableContent	Line 118: <code>const originalPreviewSectionHasRenderableContent = typeof previewSectionHasRenderableContent === "function" ? previewSectionHasRenderableContent : null;</code>
rich	Line 140: <code>const rich = extractRichSummary(editor);</code>
title	Line 154: <code>const title = pendingEditor.title "";</code>
description	Line 155: <code>const description = pendingEditor.description "";</code>
isTool	Line 156: <code>const isTool = pendingEditor.type === "tool";</code>
isSimpleText	Line 157: <code>const isSimpleText = pendingEditor.type === "text-array";</code>
isSectionDetails	Line 158: <code>const isSectionDetails = pendingEditor.type === "custom-section";</code>
usesRich	Line 159: <code>const usesRich = pendingEditorUsesRichDescription();</code>
project	Line 202: <code>const project = selectedProject();</code>
type	Line 204: <code>const type = button.dataset.openEditor;</code>
mode	Line 205: <code>const mode = button.dataset.mode "add";</code>
index	Line 206: <code>const index = button.dataset.index;</code>
section	Line 208: <code>const section = (project.sections []).find((item) => item.id === button.dataset.sectionId);</code>
item	Line 209: <code>const item = mode === "edit" ? section?.items?.[Number(index)] : {};</code>
section	Line 222: <code>const section = (project.sections []).find((item) => item.id === button.dataset.sectionId);</code>
project	Line 242: <code>const project = selectedProject();</code>
title	Line 244: <code>const title = form.elements.title.value.trim();</code>
richEditor	Line 245: <code>const richEditor = form.querySelector("[data-rich-editor='pending-description']");</code>

Item	Explanation
richDescription	Line 246: const richDescription = richEditor ? extractRichSummary(richEditor) : null;
description	Line 247: const description = richEditor ? richPlainTextOrSpacer(richDescription) : form.elements.description?.value.trim() "";
section	Line 253: const section = (project.sections []).find((item) => item.id === pendingEditor.sectionId);
nextItem	Line 256: const nextItem = {
section	Line 278: const section = (project.sections []).find((item) => item.id === pendingEditor.sectionId);
section	Line 297: const section = (project.sections []).find((item) => item.id === sectionId);
parsed	Line 352: const parsed = originalParseProjectForPortfolio(project);
sectionId	Line 355: const sectionId = parsedSection.id.replace("custom:", "");
sourceSection	Line 356: const sourceSection = (project.sections []).find((section) => section.id === sectionId);
editor	Line 394: const editor = event.target.closest("[data-rich-editor]");
block	Line 395: const block = event.target.closest(".rich-block");

Event handlers and user interactions

Item	Explanation
Line 393	document.addEventListener("contextmenu", (event) => {

Function-by-function detail

hasRichContent (builder-rich-future-sections.js, lines 23-31)

hasRichContent named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 23-31 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so builder-rich-future-sections.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references some. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 24: return Boolean(rich?.blocks?.some((block) =>.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```

23:     function hasRichContent(rich) {
24:         return Boolean(rich?.blocks?.some((block) =>
25:             (block.type === "image" && block.url) ||
26:             (block.type === "formula" && block.formula) ||
27:             block.text ||
28:             block.title ||
29:             block.caption
30:         ));
31:     }

```

richPlainTextOrSpacer (builder-rich-future-sections.js, lines 33-36)

richPlainTextOrSpacer named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 33-36 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so builder-rich-future-sections.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references plainTextFromRich, trim, and hasRichContent. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 35: return text || (hasRichContent(rich) ? "\u200B" : "");.

Important local values include text at line 34. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
33:     function richPlainTextOrSpacer(rich) {
34:         const text = plainTextFromRich(rich).trim();
35:         return text || (hasRichContent(rich) ? "\u200B" : "");
36:     }
```

pendingEditorUsesRichDescription (builder-rich-future-sections.js, lines 41-43)

pendingEditorUsesRichDescription named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 41-43 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so builder-rich-future-sections.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references includes. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 42: return Boolean(editor && ["custom", "custom-section"].includes(editor.type));.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
41:     function pendingEditorUsesRichDescription(editor = pendingEditor) {
42:         return Boolean(editor && ["custom", "custom-section"].includes(editor.type));
43:     }
```

pendingEditorFolder (builder-rich-future-sections.js, lines 48-57)

pendingEditorFolder named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 48-57 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so builder-rich-future-sections.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references slugify. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 4 return statement(s). The most important visible returns are: line 49: if (!editor) return "custom-overview";; line 51: return `custom-\${slugify(editor.sectionId || "section")}-overview`;; line 54: return `custom-\${slugify(editor.sectionId || "section")}-subsection`;; line 56: return "custom-overview";.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
48:     function pendingEditorFolder(editor = pendingEditor) {
49:         if (!editor) return "custom-overview";
50:         if (editor.type === "custom-section") {
51:             return `custom-${slugify(editor.sectionId || "section")}-overview`;
52:         }
53:         if (editor.type === "custom") {
54:             return `custom-${slugify(editor.sectionId || "section")}-subsection`;
55:         }
56:         return "custom-overview";
57:     }
```

pendingEditorRichBlocks (builder-rich-future-sections.js, lines 62-65)

pendingEditorRichBlocks named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 62-65 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so builder-rich-future-sections.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references textBlocksFromPlainText, and richBlocksFromValue. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 63: if (!editor) return textBlocksFromPlainText("");; line 64: return richBlocksFromValue(editor.richDescription, editor.description || "");.

No major local variable declarations were detected in the extracted function body.

Relevant source excerpt:

```
62:     function pendingEditorRichBlocks(editor = pendingEditor) {
63:         if (!editor) return textBlocksFromPlainText("");
64:         return richBlocksFromValue(editor.richDescription, editor.description || "");
65:     }
```

renderPendingRichDescriptionEditor (builder-rich-future-sections.js, lines 72-103)

renderPendingRichDescriptionEditor renders ui, markup, or display output. In this file, it appears around lines 72-103 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so builder-rich-future-sections.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references pendingEditorFolder, and escapeHtml. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 75: return `.

Important local values include label at line 73; folder at line 74. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
72:     function renderPendingRichDescriptionEditor(editor = pendingEditor) {
73:         const label = editor?.type === "custom-section" ? "Section overview" : "Subsection overview";
74:         const folder = pendingEditorFolder(editor);
75:         return `
76: <div class="pending-rich-description rich-design-summary-field">
77:   <div class="summary-builder-heading">
78:     <span>${label}</span>
79:   </div>
80:   <div class="rich-summary-panel">
81:     <div class="rich-summary-toolbar" aria-label="${escapeHtml(label)} tools">
82:       <button type="button" data-rich-action="font-small">Small</button>
83:       <button type="button" data-rich-action="font-normal">Normal</button>
84:       <button type="button" data-rich-action="font-large">Large</button>
85:       <button type="button" data-rich-action="align-left">Left</button>
86:       <button type="button" data-rich-action="align-center">Center</button>
87:       <button type="button" data-rich-action="align-right">Right</button>
88:       <button type="button" data-rich-action="add-image">Add image</button>
89:       <button type="button" data-rich-action="add-formula">Add formula</button>
90:       <button type="button" data-rich-action="edit-block">Edit selected</button>
91:       <button class="danger-icon" type="button" data-rich-action="delete-block">Delete
selected</button>
92:     </div>
93:     <div
94:       class="rich-summary-editor"
95:       data-rich-editor="pending-description"
96:       data-rich-folder="${escapeHtml(folder)}"
97:       data-placeholder="Write the overview here. You can paste images directly into this area."
98:       aria-label="${escapeHtml(label)} rich editor"
99:     ></div>
100:   </div>
101: </div>
102: `;
103:     }
```

section (builder-rich-future-sections.js, lines 208-209)

section named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 208-209 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so builder-rich-future-sections.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

Important local values include section at line 208; item at line 209. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
208:         const section = (project.sections || []).find((item) => item.id ===
button.dataset.sectionId);
```

```
209:         const item = mode === "edit" ? section?.items?.[Number(index)] : {};
```

section (builder-rich-future-sections.js, lines 222-230)

section named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 222-230 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so builder-rich-future-sections.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find, and openPendingEditor. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It does not mainly return a value; its output is the changed screen state or DOM it updates.

Important local values include section at line 222. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
222:         const section = (project.sections || []).find((item) => item.id ===
button.dataset.sectionId);
223:         openPendingEditor({
224:             type,
225:             mode: "edit",
226:             sectionId: button.dataset.sectionId,
227:             title: section?.title || "",
228:             description: section?.description || "",
229:             richDescription: section?.richDescription || null
230:         });
```

section (builder-rich-future-sections.js, lines 253-262)

section named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 253-262 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so builder-rich-future-sections.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 254: if (!section) return;.

Important local values include section at line 253; nextItem at line 256. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
253:         const section = (project.sections || []).find((item) => item.id ===
pendingEditor.sectionId);
254:         if (!section) return;
255:         section.items = section.items || [];
256:         const nextItem = {
257:             title,
258:             description,
259:             richDescription,
260:             type: "Text",
261:             status: "draft"
262:         };
```

section (builder-rich-future-sections.js, lines 278-291)

section named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 278-291 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so builder-rich-future-sections.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find, setStatus, scheduleAutosave, renderAll, and originalSavePendingEditor. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 3 return statement(s). The most important visible returns are: line 279: if (!section) return;; line 287: return;; line 289: return originalSavePendingEditor(form);.

Important local values include section at line 278. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
278:         const section = (project.sections || []).find((item) => item.id ===
pendingEditor.sectionId);
279:         if (!section) return;
280:         section.title = title;
281:         section.description = description;
282:         section.richDescription = richDescription;
283:         pendingEditor = null;
284:         setStatus("Saved in the project editor. Click Save project to include this version in the
portfolio preview.");
285:         scheduleAutosave();
286:         renderAll();
287:         return;
288:     }
289:     return originalSavePendingEditor(form);
290: };
291: }
```

section (builder-rich-future-sections.js, lines 297-301)

section named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 297-301 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so builder-rich-future-sections.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find, and escapeHtml. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 2 return statement(s). The most important visible returns are: line 298: if (!section) return `

Important local values include section at line 297. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
297:         const section = (project.sections || []).find((item) => item.id === sectionId);
298:         if (!section) return `
```

sourceSection (builder-rich-future-sections.js, lines 356-360)

sourceSection named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 356-360 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or top-level documentation.

This function is needed so builder-rich-future-sections.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references find, hasRichContent, clone, and richPlainTextOrSpacer. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has no obvious return statement in the extracted body; it likely completes through state changes, helper calls, or event flow.

Important local values include sourceSection at line 356. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
356:         const sourceSection = (project.sections || []).find((section) => section.id ===
sectionId);
357:         if (hasRichContent(sourceSection?.richDescription)) {
358:             parsedSection.rich = clone(sourceSection.richDescription);
359:             parsedSection.description = sourceSection.description ||
richPlainTextOrSpacer(sourceSection.richDescription);
360:         }
```

improveRichContextMenu (builder-rich-future-sections.js, lines 392-405)

improveRichContextMenu named function in the repository. inspect the surrounding lines to learn its exact inputs and outputs. In this file, it appears around lines 392-405 and is part of the repository root. usually an entry file, app config, public website file, package manifest, or

top-level documentation.

This function is needed so builder-rich-future-sections.js can keep that responsibility in one named place instead of scattering it through unrelated event handlers or route code.

It calls or references addEventListener, closest, preventDefault, selectRichBlock, currentRichBlock, and syncRichContextMenuControls. Endpoint references found inside it are no direct /api endpoint strings detected. Browser storage keys found inside it are no local/session storage keys detected.

It has 1 return statement(s). The most important visible returns are: line 396: if (!editor && !block) return;.

Important local values include editor at line 394; block at line 395. These variables show what the function is assembling, checking, or handing to another layer.

Relevant source excerpt:

```
392:     function improveRichContextMenu() {
393:         document.addEventListener("contextmenu", (event) => {
394:             const editor = event.target.closest("[data-rich-editor]");
395:             const block = event.target.closest(".rich-block");
396:             if (!editor && !block) return;
397:             event.preventDefault();
398:             activeSummaryEditor = editor || block.closest("[data-rich-editor]");
399:             selectRichBlock(block || currentRichBlock(activeSummaryEditor));
400:             syncRichContextMenuControls(activeSummaryBlock);
401:             summaryContextMenu.style.left = `${event.clientX}px`;
402:             summaryContextMenu.style.top = `${event.clientY}px`;
403:             summaryContextMenu.hidden = false;
404:         }, true);
405:     }
```

Representative opening snippet

```
1: // JavaScript source code
2: (() => {
3:     "use strict";
4:
5:     /**
6:      * Extend the portfolio builder with rich description editors for custom
7:      * sections and subsections. This patch adds a new rich editor to the
8:      * pending editor for custom items so that images, formulas and other
9:      * formatted content can be used in overviews of future sections. It
10:     * hooks into existing helper functions such as `populateRichEditor`
11:     * and `saveRichEditorToProject` by overriding them at runtime. To
12:     * avoid interfering with built-in behaviour, everything is wrapped
13:     * inside an IIFE and existing functions are stored before being
14:     * redefined. If a function is not present in the current context
15:     * (because the builder script changed), the patch simply does
16:     * nothing.
17:     */
18:
19:     // Guard against applying the patch multiple times.
20:     if (window.builderRichFutureSectionsPatchLoaded) return;
21:
22:     // Helpers to detect whether an object has meaningful rich content.
23:     function hasRichContent(rich) {
24:         return Boolean(rich?.blocks?.some((block) =>
25:             (block.type === "image" && block.url) ||
26:             (block.type === "formula" && block.formula) ||
27:             block.text ||
28:             block.title ||
29:             block.caption
30:         ));
31:     }
32:
33:     function richPlainTextOrSpacer(rich) {
34:         const text = plainTextFromRich(rich).trim();
35:         return text || (hasRichContent(rich) ? "\u200B" : "");
36:     }
37:
38:     // Determine whether the pending editor should use a rich editor. All
39:     // custom sections and subsections use rich editors. Tools, design
40:     // objects and other built-in forms continue to use plain text areas.
41:     function pendingEditorUsesRichDescription(editor = pendingEditor) {
42:         return Boolean(editor && ["custom", "custom-section"].includes(editor.type));
43:     }
44:
45:     // Build a folder name for images uploaded via the pending editor. For
46:     // custom sections and subsections, use a prefix based on the section
47:     // id so that uploaded images are organised under docs/<project>/<folder>/.
48:     function pendingEditorFolder(editor = pendingEditor) {
49:         if (!editor) return "custom-overview";
50:         if (editor.type === "custom-section") {
51:             return `custom-${slugify(editor.sectionId || "section")}-overview`;
52:         }
53:         if (editor.type === "custom") {
54:             return `custom-${slugify(editor.sectionId || "section")}-subsection`;
```



```

55:     }
56:     return "custom-overview";
57: }
58:
59: // Derive blocks from a rich value or fallback plain text. When the
60: // editor is empty, provide a single text block so that the content
61: // area is editable.
62: function pendingEditorRichBlocks(editor = pendingEditor) {
63:     if (!editor) return textBlocksFromPlainText("");
64:     return richBlocksFromValue(editor.richDescription, editor.description || "");
65: }
66:
67: // Render the rich description field for the pending editor. This
68: // function builds a UI similar to other rich editors in the builder,
69: // including a toolbar and editable area with appropriate data
70: // attributes. The label varies depending on whether we're editing
71: // a section or a subsection.
72: function renderPendingRichDescriptionEditor(editor = pendingEditor) {

```